The value of a primitive recursive term at the infinite point

a machine-checked computability theorem, from the trace alone

Abstract

Let f be a primitive recursive term, read as a monotone map on lazy naturals. We prove, constructively and by a machine-checked argument, that $f(S^\omega(\bot), \ldots, S^\omega(\bot))$ is *computable*: a total function produces an explicit element of the domain, and that element is the least upper bound of the diagonal chain $f(S^m(\bot), \ldots, S^m(\bot))$. The proof is carried out from a single invariant on the term's *trace* — the record of which variable the computation demands at each step and what it has produced — rather than from the ultimate-obstination property, which is not used anywhere and which we note does not follow from the trace invariant either. We then carry the same analysis to mutual recursion, where the ultimate-obstination property is outright false but the trace-level statement is not: a block of simultaneously defined functions has a trace of the same kind, that trace denotes the block, and for two functions the verdict follows from the step terms alone.

1 The domain, and primitive recursive terms

Let D be the domain of lazy naturals: its elements are $S^k(\bot)$ for $k \in \mathbb{N}$, $S^k(0)$ for $k \in \mathbb{N}$, and $S^\omega(\bot)$. The order is

$$S^j(\bot) \le S^k(\bot) \iff j \le k, \qquad S^j(\bot) \le S^k(0) \iff j \le k, \qquad S^j(\bot) \le S^\omega(\bot),$$

and $S^k(0)$, $S^\omega(\bot)$ are maximal. In particular

$$S^k(0) \not\le S^\omega(\bot), \tag{1}$$

the two maximal shapes being incomparable: a computation that has already produced a 0 is not an approximation of $S^\omega(\bot)$. Write $F \subset D$ for the *finite* elements, i.e. everything but $S^\omega(\bot)$. Say $x \in D$ is *incomplete* if it has the form $S^k(\bot)$ (so $S^\omega(\bot)$ is incomplete but not finite), *complete* if it is some $S^k(0)$, and write $\mathrm{ht}(x) = k$ for the number of successors of a finite x. Note that a complete element is maximal among finite ones: if x is complete and $x \le y$ with y finite, then $y = x$.

A primitive recursive term q of arity n is interpreted on finite tuples,

$$\mathrm{ev}(q) : F^n \longrightarrow F,$$

by the evident clauses; $\mathrm{ev}(q)$ is monotone. The extension to all of D^n is by continuity, and it is that extension we wish to *evaluate*.

2 The trace, and the invariant MP1

2.1 What a computation looks like from outside

Take a term $t(x_1, \ldots, x_a)$, substitute $\bot$ for every variable, and ask for its normal form. Either it is already a numeral, or it is $S^{n_0}(u_0)$ where u_0 is stuck: it needs a variable, say x_{i_0}. Replace that

variable by $S(x_{i_0})$ — the least thing that could unstick it — and repeat. What comes out is a sequence

$$(n_0, i_0),\ (n_1, i_1),\ (n_2, i_2),\ \dots$$

where n_p is the number of successors produced after p substitutions and i_p is the variable needed next. Equivalently, recording how deep one has gone in each variable rather than which was taken last, it is a sequence

$$\big(n_p;\, a_{1p}, \dots, a_{ap}\big), \qquad a_{1p} + \cdots + a_{ap} = p.$$

This sequence has a *remarkable property*: it determines the value of t at every finite point at once. For

$$t\big(S^{l_1}(\bot), \dots, S^{l_a}(\bot)\big) \quad \text{has exactly } n_p \text{ successors, where } p \text{ is maximal with } a_{ip} \le l_i \text{ for all } i.$$

One does not re-run the term at each point; one reads off the largest prefix of the sequence the point can pay for. Everything below is a consequence of that one observation, applied at increasingly complicated families of points.

2.2 The trace

We record the sequence, and a little more. A *trace* of arity a is either a leaf $\langle v \rangle$ carrying a value $v \in F$, or a node consisting of

- a *walk* $iv(0), iv(1), \dots$ with $iv(k) < a$ — the variable demanded at step k;
- a *value sequence* $ov(0), ov(1), \dots$ in F — the value the term has when its replay sticks at step k;
- for each coordinate $c < a$ and each $v \in \mathbb{N}$, a *continuation*, itself a trace of arity $a - 1$.

The walk generates its own levels: step k raises coordinate $iv(k)$ by one and leaves the others alone, so with

$$lv(c, 0) = 0, \qquad lv(c, k{+}1) = \begin{cases} lv(c,k) + 1 & c = iv(k), \\ lv(c,k) & \text{otherwise}, \end{cases} \qquad \sum_c lv(c,k) = k.$$

After k steps the walk stands at $\big(S^{lv(0,k)}(\bot), \dots\big)$, and $ov(k)$ is the value there: the pair (iv, ov) is exactly the sequence of §2.1, with $n_p = \mathrm{ht}(ov(p))$ and $a_{ip} = lv(i,p)$. Recording the whole $ov(k)$ and not only its height is essential: a height cannot tell $S^k(0)$ from $S^k(\bot)$, and $\mathrm{prec}(\mathrm{zerf}, \mathrm{zerf})$ — which is $\bot$ at $\bot$ and the numeral 0 above it — refutes any height-only version.

Replay. A trace is evaluated at an actual tuple X of finite elements by running the walk against the heights X has. Let

$$\mathsf{stick}(X) \;=\; \text{the first step } k \text{ at which the walk asks for a level that } X \text{ does not have.}$$

This is a bounded search: the walk spends one level per step, so $1 + \sum_c \mathrm{ht}(X(c))$ steps of fuel suffice. Writing $k = \mathsf{stick}(X)$, $i = iv(k)$ and $v = \mathrm{ht}(X(i))$, the value is given by a three-way case:

$$\mathsf{sem}\langle v \rangle(X) = v, \qquad \mathsf{sem}(\mathrm{node})(X) = \begin{cases} ov(k) & ov(k) \text{ complete}, \\ ov(k) & X(i) \text{ incomplete}, \\ \mathsf{sem}(\text{continuation at } i, v)\ (X \text{ with } i \text{ deleted}) & X(i) \text{ complete}. \end{cases}$$

The three clauses say: the computation has already answered; or it is blocked on a coordinate that is still $S^j(\bot)$, and $ov(k)$ is the answer; or that coordinate has turned out to be a numeral, in which case it is frozen into the term and the computation continues at arity $a-1$. The last clause is why a trace is a tree, and why every argument below which involves completeness is a recursion on the *arity*.

Two consequences are used constantly. First, *if no coordinate of X is complete the third clause never fires*, so

$$\mathsf{sem}(T)(X) \;=\; ov\big(\mathsf{stick}(X)\big) \tag{2}$$

— the whole semantics collapses to a single lookup in the value sequence. Second, since a walk spends at most one level per step, $lv(c,k) \le k$; combined with the fact that it sticks exactly when the level it wants is the one the coordinate does not have, this gives the dichotomy of §2.4.

The trace of a term. The trace is built by induction on the term, and — this is the point of building it rather than postulating its shape — it is proved to *denote* that term: $\mathsf{sem}(T_q)(X) = \mathrm{ev}(q)(X)$ for every X of the right length. So the trace is not a separate model to be related to $\mathrm{ev}(q)$ afterwards; it computes $\mathrm{ev}(q)$, and iv, ov are a decomposition of that computation.

Remark 1 (provenance, and what this trace is *not*)**.** The sequence (n_p, i_p) of §2.1, together with the observation that it determines the value of the term at every finite point at once, is the notion of trace of an email from Coquand to Colson of 1991; the direct constructive proof of Colson's ultimate obstination theorem that came out of it is *Une preuve directe du théorème d'ultime obstination*, C. R. Acad. Sci. Sér. I **314** (1992). The trace of §2.2 is that notion, with the whole value $ov(k)$ recorded in place of its height alone, and with continuations added for the coordinates that go complete.

It is *not* the notion of trace of R. David (*On the asymptotic behaviour of primitive recursive algorithms*, Theoret. Comput. Sci. **266** (2001) 159–193, and *Decidability results for primitive recursive algorithms*, Theoret. Comput. Sci. **300** (2003) 477–504, the paper that extends the constructive result to mutual recursion), although the two are aimed at the same theorem. There a trace is a pair (e, λ): an element e of the domain *together with a labelling* assigning to each accessible cell of e a word of tokens x_n, recording which cell of which argument was used to produce that cell. A term denotes a continuous map from traces to traces, the arguments being themselves traces. So David's trace is a computation *at a given tuple of arguments*, decorated with provenance — in his own reading, a branch of a Berry–Curien sequential algorithm.

The trace used here carries no provenance and is attached to the term alone. It is the record of the canonical increasing walk that starts at $(\bot, \dots, \bot)$ and raises, at each step, exactly the coordinate the computation is stuck on, together with the value produced at each step; the value at an arbitrary tuple is recovered by *replay* (§2.2), not by reading a label. The two therefore record different things, and neither is here defined from the other: David's is the finer datum at a fixed point, since it says which cells were used, while the present one is the finer datum along the increasing family, since it keeps the whole sequence of values — distinguishing $S^k(0)$ from $S^k(\bot)$ — and descends into a continuation when a coordinate becomes a numeral.

One derived datum does cross, and it is the one his treatment of mutual recursion runs on. What that treatment uses of a trace is the static *calls graph* — which argument occurrence a recursive call reads — which is available to him from the syntax, the call sitting in the term. Here the demand is not a label but a computation, a function of the incoming tuple; unwinding it, however, gives

$$\text{the coordinate demanded at } X \;=\; iv\big(\mathsf{stick}(X)\big),$$

provided $ov(\mathsf{stick}(X))$ and the demanded coordinate of X are both still incomplete — the tuple entering only through $\mathsf{stick}(X)$ and through those two completeness questions. So the demanded

coordinate is the walk's own label; and by clause (i) of Definition 1 that label is eventually constant, so with N_i its threshold, $\mathrm{nx}(i) = iv_i(N_i)$ is David's nx — read off the invariant rather than off the syntax. This is not a translation of his traces into these: the labelling of every accessible cell is not recovered, and the identification holds only past the threshold and while nothing has completed, which is Colson's obstinate regime. It is enough to carry his unrolling, and §5 records that it is also avoidable.

The theorem has also been proved on the concrete-data-structure side itself, by P.-L. Curien (*Sequential interactive behaviour of recursive program schemes*, talk, Tallinn, December 2017): a scheme is interpreted as a sequential algorithm, and ultimate obstination says that an infinite branch of it calls infinitely often on exactly one argument. The proof runs an abstract machine for $\mathrm{rec}(g, h)$ whose states carry a stack of pending recursive calls, and turns on a *race* between the outputs a branch of the step term issues and the point at which it asks for its recursive argument: unless enough outputs have been issued first, the machine revisits a prefix it has already run, contributes no new call to any argument, and pushes stacks of unbounded depth — so the branch is obstinate with the recursion argument as its index. That race is the operational form of the verdict clause of Definition 1: enough successors must be produced, and soon enough, relative to the demands. The reading is close to David's, as one would expect of two accounts written in the same model.

2.3 The invariant

Definition 1 (MP1)**.** A value sequence ov has a *verdict* when

- either $ov(k)$ is complete for some k — and then, completeness being maximal and ov monotone, ov is constant from k on;
- or no $ov(k)$ is complete, and there is a threshold past which $\mathrm{ht} \circ ov$ is either constant or strictly increasing.

A trace satisfies MP1 when at every node of it

(i) the walk iv is eventually constant, and

(ii) the value sequence ov has a verdict,

and every continuation satisfies MP1 as well.

Clause (i) is ultimate obstination read along the walk: from some step on the computation demands one variable and never changes its mind. Clause (ii) is the three-way shape of the value along that variable.

Two remarks on the exact form. First, the completeness disjunct has to be kept separate: it is the case where the computation answers, and no statement about heights can express it. Second, in the incomplete case one asks for *constant or strictly increasing*, not merely for the weaker dichotomy "bounded or unbounded". The weaker one is what the theorem of §3 consumes, but it is not closed under recursion: for a step whose height may drop back, the orbit's verdict is equivalent to the limited principle of omniscience. Constant-or-strictly-increasing repairs this, and one sees why in the recursion clause — for a strictly increasing step a single comparison decides the orbit, since $x \mapsto \varphi(x)$ either fixes a point or moves it upward for ever.

Theorem 1. *The trace of every primitive recursive term satisfies* MP1*.*

2.4 The engine: the stuck coordinate

Everything in the proof of Theorem 1 runs on one observation. Suppose a trace is fed a family $X_0 \le X_1 \le \dots$ of argument tuples and let $N_t = \mathsf{stick}(X_t)$, which is monotone. At time t the replay is stuck at step N_t on the coordinate $d = iv(N_t)$, and *the level it needs there is exactly the height d offers*: the levels a walk has consumed on the way stay below what was available, and sticking says the one it wants is not. Hence:

> *if d never grows again, the replay is stuck at N_t for ever; if d grows, that is precisely the level it was waiting for, and the replay advances.*

There is no third possibility. So N strictly increases until it freezes, and it freezes for good — and *which* of the two happens is decided by the regime of the stuck coordinate, which the induction hypothesis supplies. Deciding it is moreover a *bounded* search: if N has not frozen by the threshold of clause (i), then N has passed that threshold, so from there the stuck coordinate is the single eventual demand, whose regime is already known.

Given (2), this settles the value: if the replay freezes, the value is constant; if it strictly increases, the fed trace's own verdict transfers along it verbatim. The remaining case is a coordinate going *complete*, and there the replay freezes too — a complete value is maximal, so the coordinate never moves again — which makes the continuation descended into a *fixed* smaller trace, and the whole argument recurses at arity $a - 1$. At most a descents happen.

2.5 Composition

Let $f(\vec{x}) = g\big(h_1(\vec{x}), \dots, h_p(\vec{x})\big)$. The arguments read the *same* variables, so they cannot be stepped independently: driving them one at a time charges a shared level twice and makes the composite stick too early. (Concretely, $f(x) = g(x, x)$ with g the identity in its first argument then answers $S(\bot)$ at $S^2(\bot)$.) The state must therefore be the composite's own level vector, with each argument *replayed against it* — which is exactly the remarkable property of §2.1 applied to each h_i at the current $(l_1, \dots, l_a)$. Sharing is then automatic: raising one level advances every argument that was waiting for it, at no extra cost.

The composite demands the level that the argument g is currently blocked on needs; so the selected argument advances by at least one replay step per stage. That *drive* is what carries an argument's own verdict, which is stated in its private replay depth, over to the composite's clock. Together with the fact that the composite's demand is eventually constant — say it waits on argument J for ever — the three cases of §2.4 apply with g as the fed trace and the argument values as the family:

- if J's value settles, the composite's value is constant, and this needs nothing but agreement at the coordinate g is waiting on;
- if J's value keeps growing, g's replay strictly increases and g's own verdict transfers;
- if J's value becomes a numeral, g's replay freezes, the continuation is fixed, and the argument recurses at smaller arity.

2.6 Primitive recursion

Let $f(\bot, \vec{y}) = \bot$ and $f(S^{j+1}(\bot), \vec{y}) = g\big(S^j(\bot),\, f(S^j(\bot), \vec{y}),\, \vec{y}\big)$. Along f's own walk, past the threshold of clause (i) exactly one coordinate grows, and there are two shapes.

The recursion argument grows. Then the parameters are frozen and f's value sequence is the *chain*

$$V_0 = \bot, \qquad V_{j+1} = g\big(S^j(\bot),\, V_j,\, \vec{y}\big).$$

While V_j is incomplete every coordinate of g's argument tuple is still of the form $S^{\cdot}(\bot)$, so (2) applies and

$$V_{j+1} \;=\; ov_g(N_j), \qquad N_j = g\text{'s replay depth at depth } j.$$

This is the situation of §2.4, and here it has a sharp reading. Writing g's sequence as $(n_p;\, x_p, y_p, z_p)$ — n_p successors after p steps, having gone x_p deep in the recursion argument, y_p in the recursive value, z_p in the parameters — the criterion is

f loops in its first argument if and only if there is p with $y(p{+}1) = y(p){+}1$ and $n_p \le y_p$,

and then f's own sequence is ultimately $(n_p;\, x, z), (n_p;\, x{+}1, z), \ldots$ — constant number of successors — while otherwise it is essentially g's. The two halves of the criterion say exactly that step p demands the recursive value and that the successors produced do not exceed the depth already consumed of it: that is, that g's replay *stalls* on the recursive value.

Two facts make this decidable. First, *a stall is never on the recursion argument*: that coordinate grows by one per depth all by itself, whereas the levels consumed on the way to the sticking step are below what was available one depth earlier. Second, a stall on anything else is *permanent*: a parameter never moves again, and a stall pins the recursive value, so the coordinate it waits on does not move either. Hence either the replay strictly increases at every depth — and g's verdict transfers — or it freezes and the height is constant. Which one is settled by a bounded search, exactly as in §2.4, and here clause (i) for g is what bounds it: past g's own threshold the stalled coordinate must be g's eventual demand, and each of the three possibilities for that demand is decided.

A parameter grows. Then the recursion depth is frozen at some c, and f along the growing parameter is g applied c times over. An induction on c does it: layer $c+1$ is g fed the family

$$\big(S^c(\bot),\ \text{layer } c,\ S^{l+t}(\bot), \ldots\big),$$

in which the recursion argument and the frozen parameters never move, the growing parameter grows at every step, and the middle coordinate is whatever the induction hypothesis says. That is precisely the hypothesis of the general lemma of §2.4, so each layer is one application of it.

Remark 2. The direction is worth stating explicitly, because an earlier version of this development went the other way. The trace argument above does *not* consume the ultimate-obstination property; it is proved from the trace by induction on the term, and the ultimate-obstination property is not used anywhere in it.

3 The theorem

Fix an arity n and write

$$\Delta_m \;=\; (S^m(\bot), \ldots, S^m(\bot)) \in F^{\,n}, \qquad \Delta_\infty \;=\; (S^\omega(\bot), \ldots, S^\omega(\bot)) \in D^n.$$

The chain $c_m := \mathrm{ev}(q)(\Delta_m)$ is monotone in m by monotonicity of $\mathrm{ev}(q)$.

Theorem 2. *For every primitive recursive q of arity n one can compute an element $\mathsf{val}(q) \in D$ which is the least upper bound of $(c_m)_{m\in\mathbb{N}}$. In particular $q \mapsto \mathsf{val}(q)$ is total, so $f(S^\omega(\bot), \ldots, S^\omega(\bot))$ is computable, uniformly in q.*

The all-infinite point is the *easy* point, and the proof is three lines of the machinery of §2.

Lemma 1 (the chain is a lookup)**.** *$c_m = ov(N_m)$, where N_m is the replay depth of the trace against the level vector $(m, \dots, m)$.*

Proof. No coordinate of Δ_m is complete, so (2) applies; and the trace denotes $\mathrm{ev}(q)$. □

Lemma 2 (the lookup is cofinal)**.** *$N_m \geq m$.*

Proof. The walk spends at most one level per step, so at step $k < m$ it has consumed at most $k < m$ levels of the coordinate it is about to demand, and m are available. Hence it does not stick before step m. □

So the chain, reindexed, runs through the value sequence cofinally, and its least upper bound is the least upper bound of ov. The verdict of Definition 1 names that:

$$\mathsf{val}(q) \;=\; \begin{cases} ov(k) & ov \text{ complete at } k \text{ (a numeral)}, \\ S^{\mathrm{ht}(ov(k))}(\bot) & ov \text{ never complete, height constant from } k, \\ S^{\omega}(\bot) & ov \text{ never complete, height strictly increasing from } k. \end{cases}$$

Proof of Theorem 2. Complete at k. Then ov is constant from k on. For any m, $ov(N_m) \leq ov(k)$: either $N_m \leq k$ and this is monotonicity, or $N_m \geq k$ and the two are equal. So $ov(k)$ is an upper bound, and by Lemma 2 it is attained at $m = k$, hence least.

Never complete, height constant from k. Every c_m is $S^{\mathrm{ht}(ov(N_m))}(\bot)$, and $\mathrm{ht}(ov(N_m)) \leq \mathrm{ht}(ov(k))$ by the same split. Attained at $m = k$, again by Lemma 2.

Never complete, height strictly increasing from k. Each c_m is incomplete, hence below $S^{\omega}(\bot)$, so $S^{\omega}(\bot)$ is an upper bound. It is the least one: a strictly increasing height is unbounded, so for any finite d — of either shape — Lemma 2 produces an m with $\mathrm{ht}(c_m) > \mathrm{ht}(d)$, and d is not an upper bound. □

Remark 3. Only Lemma 1 uses that *every* coordinate of the point is infinite, and it uses only that none is complete and none is finite — so nothing descends and nothing blocks. That is the whole reason the argument at Δ_∞ is so much shorter than at a general point.

Remark 4 (what MP1 does not give)**.** It does *not* give the ultimate-obstination property. The leaf trace $\langle S^0(\bot)\rangle$ of arity one satisfies MP1 vacuously and denotes the constant $\bot$, whose ultimate-obstination property fails at the point (0): at an all-complete point the property leaves no room — its second case wants a coordinate that is incomplete and finite, its third wants one that is $S^{\omega}(\bot)$ — so its first case must hold and the value must be a numeral, which a constant $S^m(\bot)$ is not. The missing ingredient is *totality*, that a term applied to numerals returns a numeral; that is a separate (easy) induction on the term, and it is not needed for Theorem 2.

4 The formalisation

All of the above is formalised in Agda and checked with `--safe --without-K --exact-split`, with no postulate, no hole and no termination pragma.

- The trace, its semantics and its levels, and the proof that the trace of a term denotes the term: `TraceDef`, `ReplayLv`, `TrSat`, `TrDen`, `TrTerm`. The collapse (2) is `TrSat.sem-bot`.

- The invariant and its two clauses: `TrMP1`. Composition: `TrComp`, `TrCompNG`, `TrCompSel`, `TrSelStab` for the index clause, and `TrCompVal`, `TrCompVerdict`, `TrCompMP1` for the value clause. Recursion: `TrPrec`, `TrPrecIvPMP` for the index clause, and `TrPrecChain`, `TrPrecStall`, `TrPrecPhi`, `TrPrecParPhi`, `TrPrecOvP` for the value clause. The general lemma of §2.4 is `TrFeedR` (with `TrCompVerdict` as the special case in which the demand is known in advance to settle).
- Theorem 1 is `TrTermMP1`, and Theorem 2 is `PRInfMP1`. Neither uses the ultimate-obstination property; the only thing either takes from the development of that property is its well-formedness *predicate*.
- Remark 4 is `TrUOFail`, and the totality it refers to is `PRTot`.
- For §5: the trace of a block and the proof that it denotes the block are `TrMrec` and `TrMrecEval`; the verdict for two functions is `BlkR2Value`. The calls graph read off the walk is `TrMrecCalls`, the unary unrolling and its iteration law `TrMrecFix`, and the unconditional r-ary one `TrMrecPsi`.

The proof *runs*: a companion file checks by evaluation that

$$\begin{array}{lcl} \mathrm{zerf}(S^\omega(\bot)) & = & 0, \\ \mathrm{proj}_0(S^\omega(\bot)) \;=\; \mathrm{succ}(S^\omega(\bot)) & = & S^\omega(\bot), \\ E(S^\omega(\bot)) & = & 0 \quad \text{where } E = \mathrm{prec}(\mathrm{zerf}, \mathrm{zerf}). \end{array}$$

E is the informative one: $E(\bot) = \bot$ but $E(S^{m+1}(\bot)) = 0$, so the chain is $\bot, 0, 0, \ldots$ and the least upper bound is a numeral, at a point that is infinite.

5 Mutual recursion

Consider now a block of r functions defined by simultaneous primitive recursion,

$$f_j(S(x), \vec{y}) \;=\; h_j\big(x,\; f_1(x, \vec{y}), \ldots, f_r(x, \vec{y}),\; \vec{y}\big), \qquad 1 \le j \le r, \tag{3}$$

with the h_j already given.

For such a block the ultimate-obstination property is *false*: there is a two-function block whose value along the diagonal has height $\lfloor m/2 \rfloor$, which is neither constant nor strictly increasing, so the shape the property's third case demands cannot be met. This does not touch Theorem 2, which is proved from the trace and MP1 alone and nowhere from that property; and $\lfloor m/2 \rfloor$ is unbounded, so the value of that very block at Δ_∞ is $S^\omega(\bot)$, and is computed. What a block needs is MP1, not ultimate obstination.

Sections 2.4–2.6 transfer unchanged: they are about a trace fed a monotone family, and nothing in them mentions how the function was defined. The stuck-coordinate dichotomy, the bounded search that decides it, and the descent at a coordinate that goes complete therefore apply to a block's step terms as they stand, and Lemmas 1 and 2 are facts about Δ_∞ and the order on D.

The trace of a block. A block has a trace of exactly the kind of §2.2: it is the trace of primitive recursion with its layer a vector, the block unfolding on the numeral depth rather than on a subterm. It carries value sequences and continuations as a single term's trace does, and, like it, is proved to denote what it is the trace of — its semantics is the block's evaluation, component by component. So nothing in §2 has to be restated for blocks; only the object it is applied to changes.

Recording values rather than heights is what makes that true. An account of the same dynamics that keeps only how tall each component is at each unfolding stays correct as long as no component has answered, and fails as soon as one does: for $f_1 = 0$ and $f_2 = f_1 + S(f_2)$ the real f_2 climbs by one per unfolding, so its value is $S^\omega(\bot)$, while the height-only dynamics freezes it at $\bot$. The reason is the third clause of the semantics of §2.2 — a coordinate that has gone complete makes the computation descend into a continuation — and a height cannot see a descent.

Step terms, not arbitrary traces. The invariant does not transfer for arbitrary traces. There is a pair of traces satisfying the corresponding hypotheses for which the bare bounded-or-unbounded dichotomy for the block implies the limited principle of omniscience; classically the same point reads that "is the height of $f(S^m(\bot))$ bounded in m?" is Π^0_1-hard. That example does not come from a block of terms. Its step height is $b(n) + n$ for an arbitrary binary b, and what makes the orbit hard is exactly that b may drop from 1 to 0; a step term's height is constant or strictly increasing past a threshold that the proof of $\mathsf{MP1}$ produces, and $b(n) + n$ admits such a threshold only when b never drops after it — which already decides the principle. So closure fails over arbitrary traces, and an argument for a block must use the step terms; no primitive recursive block is exhibited whose value at Δ_∞ is uncomputable.

Two functions. For $r = 2$ the verdict for the block follows from the invariants its step terms already carry, with no obstinacy assumed anywhere. Each component is decided separately to be obstinate or to answer, and decided from a supremum that is *attained* rather than merely bounded — a bound on the replay depth does not say whether a component ever answers. A component that has answered is fixed for ever, so the block descends to a system of the same shape at one lower arity, and the same argument applies to it.

The unrolling, and David's calls graph. A block's depth sequence is the orbit of a single map, and there are two ways to say so, differing exactly in whether the calls graph of Remark 1 is used.

Substituting into the one coordinate that graph names gives David's unary Φ_i, and with it his iteration law: the value after $n + p$ unfoldings is Φ_i applied to the value after n. What that costs is the hypothesis that the step term really is blocked at the named coordinate, which is the conjunction of the two obstinacy conditions of Remark 1 with the requirement that the walk has passed its threshold. The last is not free: a step term stuck on a parameter does not advance, and so does not reach it.

Filling *every* recursive coordinate at once removes the need for it. Advancing the depth coordinate by t successors — David's shift $x \mapsto x + p$, here literally S composed t times — the tuple the step term receives is the block's own t-th unfolding, coordinate by coordinate; so there is no saturation step and no demand to control, and the fixpoint law holds with nothing assumed at all. What is traded away is that the iteration is then a map on vectors of length r and not a unary one. Free and r-ary, or unary and conditional: that is the choice, and it is the only place in the block argument where the calls graph does any work.

The value of a block at the infinite point. The level-by-level verdict holds for blocks unconditionally: for every K one can decide whether the value reaches S^K, because a component's height past its threshold is a deterministic monotone iteration, decided by $K + 1$ steps. So the value of a block at the infinite point is computable *as a lazy natural*, which is the constructively correct reading of such an object. Producing it instead as an element of the three-constructor datatype D — which

builds the bounded-or-unbounded dichotomy in, $S^k(\perp)$ and $S^\omega(\perp)$ being different constructors — is what the trace of a block gives, for two functions.